

VioGen: Targeted Single-Type DRC Violation Case Generator with Agentic Orchestration for Debugging and Tool Benchmarking

ABSTRACT

High-quality layout pattern libraries are essential to physical verification tasks such as design-rule checking (DRC) tool validation, rule-deck debugging, and hotspot benchmarking. Beyond DRC-clean patterns, these tasks frequently demand violation cases that contain exactly one target failure of a specified type, location, and severity, while maintaining compliance with all non-target rules. Existing layout pattern generation methods focus exclusively on DRC-clean output and cannot produce such controlled violations due to strong geometric coupling among rules and frequent cascaded failures. We present VioGen, a framework that generates targeted single-type DRC violation cases from natural-language instructions through agentic orchestration. An LLM-based violation planner structures user intent into executable task specifications, coordinates generation and verification in a closed loop, and performs failure-aware replanning when cascaded violations are detected. A conditional flow-matching editor performs contour-aware geometric editing in normalized vectorized pattern space, where projection at each ODE step suppresses unintended rule failures, while ratio normalization enables efficient transfer across rule settings. Experiments on five representative rule categories demonstrate that VioGen achieves 93.0% average target violation success rate with only 5.3% non-target leakage and 86.2% zero-shot transfer efficiency under unseen rule thresholds, extending layout pattern generation from DRC-clean output to controllable violation-aware case construction.

1 INTRODUCTION

As semiconductor process nodes continue to shrink, physical verification flows increasingly depend on diverse layout pattern libraries for DRC tool validation [1], rule-deck debugging [2], stress testing [3], and hotspot benchmarking [4, 5]. In practice, these tasks require not only DRC-clean patterns but also carefully constructed violation cases that expose tool behavior under specific geometric corner cases [6, 7]. However, manual construction of such violation libraries is labor-intensive, error-prone, and heavily reliant on expert knowledge of layout geometry and complex rule interactions [8, 9].

Existing studies on layout pattern generation have made substantial progress in improving pattern quality and diversity. Early approaches relied primarily on rule-based construction and heuristic augmentation, producing new samples through primitive recombination or simple geometric transformations, yet these methods typically exhibit limited topological diversity and poor scalability. With the adoption of machine learning, layout pattern generation has gradually shifted toward data-driven modeling. DeePattern [8] and CUP [10] introduced autoencoder-based generation from topology matrices; LayoutTransformer [11] formulated pattern generation as sequence prediction; DiffPattern [9] applied diffusion models to improve generation diversity and realism; and ControlLayout [12] explored conditional generation with layout-level control signals. More recently, image-space methods such as PatternPaint [13] and

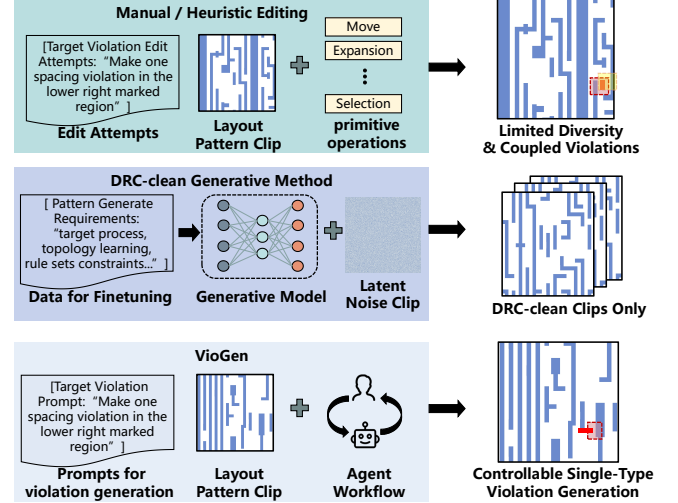


Figure 1: Comparison of three layout pattern generation paradigms: (a) manual/heuristic editing with limited diversity and coupled violations, (b) DRC-clean generative methods producing only legal clips, and (c) VioGen generating controllable single-type violations via an agent workflow.

Vario [14] attempt to capture richer geometric structures directly from pixel-level layouts. Despite these advances, all aforementioned methods are designed for DRC-clean pattern generation and do not support targeted violation generation with explicit control over rule type, location, and severity.

Targeted DRC violation generation is fundamentally more challenging than clean pattern generation. Rather than merely matching the training distribution under legality constraints, the generation process must simultaneously realize a desired target violation and preserve compliance with all non-target rules. This is particularly difficult because the geometric quantities governing different rules—such as width, spacing, area, and connectivity—are spatially interdependent: a local edit intended to activate one target rule can readily disturb the geometric margins of other rules and trigger additional non-target violations. The difficulty is further compounded for context-dependent rules such as EOL spacing, whose violation status depends on the precise location, orientation, and geometry of line-end structures. For methods operating in raster or topology space, the responsible polygon instance is not explicitly exposed, rendering precise control inherently limited.

Another challenge lies in the interface between user intent and geometric generation. In realistic verification workflows, users describe desired cases in natural language, often specifying the target rule type, preferred region, severity, and surrounding context. Translating such high-level requirements into a valid generation procedure requires task decomposition, tool invocation, verification feedback handling, and failure recovery—capabilities that extend beyond geometric editing alone. Recent advances in large language

models have demonstrated considerable potential for such multi-step task orchestration in EDA workflows [15, 16]. In the layout generation domain, ChatPattern [17] demonstrates that an LLM-based front end can effectively bridge natural-language requirements and pattern generation tools, suggesting a promising direction for controllable DRC case construction.

In this paper, we present VioGen, a framework that generates targeted single-type DRC violation cases from natural-language instructions through the coordination of an LLM-based violation planner and a conditional flow-matching violation editor. Our contributions are as follows:

- To the best of our knowledge, VioGen is the **first framework for targeted single-type DRC violation case generation** with explicit control over rule type, location, and severity. We formalize this problem and establish a corresponding evaluation protocol with six metrics covering violation success, isolation, severity precision, diversity, and transferability.
- We integrate an **LLM-based violation planner** that translates natural-language instructions into structured task specifications and performs failure-aware replanning, improving first-attempt success from 76.4% to an overall 95.6% through feedback-driven recovery.
- We develop a **conditional flow-matching violation editor** that operates in normalized vectorized pattern space with a two-bit binary direction encoding. A constraint-projected sampling scheme enforces non-target-rule feasibility at each ODE integration step, achieving 93.0% average TVSR with only 5.3% non-target leakage across five rule categories.
- We propose a **rule-ratio normalization** scheme that decouples the editing prior from absolute rule thresholds, enabling 86.2% zero-shot transfer efficiency and 90.5% few-shot recovery under unseen rule settings.

2 PRELIMINARY AND PROBLEM FORMULATION

2.1 Design Rules and Violation Types

We consider five representative DRC violation types, as illustrated in Fig. 2. The first four are basic local rules (Fig. 2(a)): minimum width, minimum spacing, minimum area, whose legality is determined by single geometric quantities, and short (Fig. 2(b)), which detects unintended connectivity between adjacent polygons. EOL spacing (Fig. 2(c)) is a context-dependent rule: a violation is triggered when two line-end structures face each other within a distance less than τ_{eol} and at least one line-end width is below τ_{eolWidth} . We use EOL spacing as the representative context-dependent rule for evaluation throughout this paper.

2.2 Conditional Flow Matching

Conditional flow matching [18, 19] is a generative modeling framework that learns a continuous transformation between two distributions by training a time-dependent velocity field. Unlike diffusion models [20, 21], which require iterative denoising from random noise, flow matching constructs a deterministic probability path between the source and target states and trains a neural network to predict the instantaneous velocity along this path. At inference

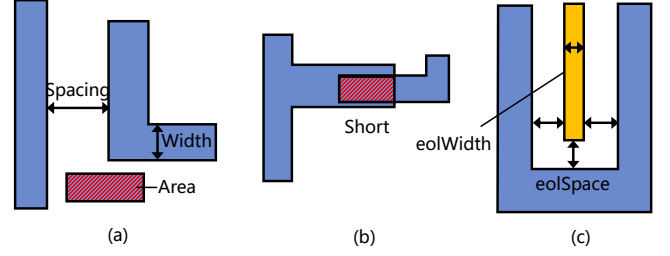


Figure 2: Representative DRC violation types: (a) basic rules (minimum spacing, width, and area), (b) short violation, and (c) end-of-line (EOL) spacing rule determined by eolWidth and eolSpace . Shaded regions indicate the violation-triggering structures.

time, the target state is obtained by solving an ordinary differential equation (ODE) from the source state, which supports deterministic sampling and naturally admits stepwise constraint enforcement at each integration step.

In this work, we adopt conditional flow matching as the backbone for the violation editor, where the source state corresponds to a clean layout pattern and the target state corresponds to the desired violation pattern in normalized vectorized space. The detailed formulation is presented in Section 3.4.

2.3 Evaluation Metrics

We adopt six metrics for targeted single-violation case generation evaluation covering violation accuracy, isolation, diversity, and cross-setting generalization.

(1) **Target Violation Success Rate (TVSR)** measures the fraction of generated patterns that successfully realize the requested target violation.

(2) **Non-Target Leakage Rate (NTLR)** measures the fraction containing at least one unintended non-target-rule failure. A lower NTLR indicates better violation isolation.

(3) **Severity Control Error (SCE)** evaluates the accuracy of severity control:

$$\text{SCE} = \frac{1}{N} \sum_{i=1}^N |s_i - s_i^*|, \quad (1)$$

where s_i^* and s_i are the requested and realized severities, respectively.

(4) **Transfer Efficiency (TE)** evaluates performance retention under unseen rule settings, defined as the ratio of TVSR on the unseen target setting to TVSR on the training setting. A TE of 100% indicates no performance degradation. We report TE in both zero-shot (no parameter update) and few-shot (limited adaptation budget) scenarios.

(5) **Entropy H_1** captures topology diversity. The topological complexity of a vectorized pattern is characterized by a tuple (C_x, C_y) , where C_x and C_y denote the numbers of unique scan lines induced by polygon edges along the x - and y -axes, respectively, each reduced by one. H_1 is defined as the Shannon entropy over the joint complexity distribution in the generated library:

$$H_1 = - \sum_i \sum_j P(C_x^{(i)}, C_y^{(j)}) \log P(C_x^{(i)}, C_y^{(j)}), \quad (2)$$

where $P(C_x^{(i)}, C_y^{(j)})$ is the probability of observing a pattern with complexity $(C_x^{(i)}, C_y^{(j)})$ in the generated library. A higher H_1 indicates that the library covers more diverse topological structures rather than concentrating on a few repeated complexity levels.

(6) Entropy H_2 captures geometry-aware diversity beyond topology complexity. For each generated pattern, a canonical signature V is constructed from the normalized start-point coordinates, direction-code sequence, edge lengths, and relative inter-polygon offsets extracted from its vectorized representation. H_2 is the Shannon entropy over the signature distribution:

$$H_2 = - \sum_k P(V^{(k)}) \log P(V^{(k)}), \quad (3)$$

where $P(V^{(k)})$ is the probability of observing signature $V^{(k)}$. Compared with H_1 , H_2 additionally reflects diversity in the actual contour-level geometric realizations: two patterns may share the same (C_x, C_y) complexity but differ substantially in polygon shapes and relative positions, which H_2 captures but H_1 does not.

2.4 Problem Formulation

Given a DRC-clean layout pattern P , a full rule set $\mathcal{R}$, and a target specification

$$c = (r^*, \Omega^*, s^*), \quad (4)$$

comprising the target rule type r^* , the target region Ω^* , and the target severity s^* , the goal of *targeted single-violation case generation* is to produce a pattern $\tilde{P}$ that violates r^* within Ω^* at severity s^* while introducing no non-target-rule failures elsewhere. For context-dependent rules, Ω^* may additionally denote the responsible contour structure, such as a candidate line-end pair for an EOL spacing violation.

The normalized severity is defined as

$$s = \frac{\tau_{r^*} - z_{r^*}(\tilde{P})}{\tau_{r^*}}, \quad (5)$$

where τ_{r^*} is the rule threshold and $z_{r^*}(\tilde{P})$ is the realized geometric quantity (spacing, width, area, or line-end distance) of the generated pattern.

Severity controllability is essential for constructing comprehensive benchmark libraries: even within a single violation type, cases of varying severity exercise different detection thresholds and classification boundaries, enabling more fine-grained evaluation of DRC tools and hotspot detectors than binary violate/pass labels alone can provide.

Unlike clean pattern generation, this task must simultaneously activate a desired target failure and suppress all collateral failures, making it a constrained geometric editing problem. An ideal method should maximize TVSR while minimizing NTLR and SCE, subject to generalization across rule settings with bounded adaptation cost.

3 VIOGEN FRAMEWORK

3.1 Workflow Overview

Figure 3 shows the overall workflow of VioGen. The workflow starts from a user request that specifies the desired violation case (1). The request is then converted into a structured task specification

$$\xi = \mathcal{A}(q) = (r^*, \Omega^*, s^*, \mathcal{B}), \quad (6)$$

Table 1: Comparison of VioGen against prior methods.

Methods	Single Type	Cond. Ctrl.	Agentic Loop	Adv. Rules	Rule Trans.	Viol. Iso.
DeePattern [8]	✗	✗	✗	✗	✗	✗
LayoutTransformer [11]	✗	✗	✗	✗	✗	✗
DiffPattern [9]	✗	✗	✗	✗	✗	✗
ChatPattern [17]	✗	✓	✓	✗	✗	✗
ControLayout [12]	✗	✓	✗	✗	✗	✗
PatternPaint [13]	✗	✓	✗	✗	✗	✗
VioGen (Ours)	✓	✓	✓	✓	✓	✓

Note. "Cond. Ctrl." denotes conditional controllability over generation (e.g., style, topology, or region for clean generators; violation type, location, and severity for VioGen). "Adv. Rules" refers to context-dependent rules such as EOL spacing.

where r^* , Ω^* , and s^* denote the target rule type, target region, and target severity, respectively, and $\mathcal{B}$ collects auxiliary generation constraints (2).

Next, VioGen constructs a diverse clean seed library from retrieved DRC-clean clips and clean patterns generated by a fine-tuned Stable Diffusion 3 model [22], followed by topology-balanced sampling (3). A selected seed pattern is then converted into the vectorized representation used by the back-end editor (4). Conditioned on ξ , the editor performs target-conditioned geometric editing to obtain an edited candidate (5)

$$\tilde{P} = \mathcal{E}(P, \xi).$$

The edited candidate is then checked by a rule verifier

$$y = \mathcal{V}(\tilde{P}, \mathcal{R}), \quad (7)$$

where $\mathcal{R}$ is the full rule set and y indicates target-rule realization together with any non-target-rule leakage (6). If verification fails, verifier feedback is used to refine the task specification and re-launch generation, forming a closed-loop workflow (7). Among these stages, vectorization is detailed in Section 3.2, specification refinement in Section 3.3, and geometric editing in Section 3.4.

3.2 Vectorized Pattern Encoding

A key design choice in VioGen is the pattern representation employed by the violation editor. Prior layout generation methods predominantly adopt the squish pattern representation [8, 9], which encodes a layout as a binary topology matrix T together with global geometry vectors Δx and Δy . Although this representation is storage-efficient and well suited to convolutional architectures, it presents two limitations for contour-level violation editing: polygon identity is absorbed into the global matrix, making it difficult to localize and edit individual polygons; and edge lengths are derived implicitly from $\Delta x/\Delta y$, providing limited control over single-edge geometric quantities.

Inspired by the sequential polygon modeling in LayoutTransformer [11], we adopt an explicit vectorized encoding that represents each pattern as an ordered polygon sequence:

$$P = [\text{SOC}, M_1, M_2, \dots, M_N, \text{EOC}], \quad (8)$$

where each polygon M_i is encoded as

$$M_i = [\text{SOP}, x_i^0, y_i^0, b_{i1}, d_{i1}, \dots, b_{iK_i}, d_{iK_i}, \text{EOP}]. \quad (9)$$

Here (x_i^0, y_i^0) is the starting point and $d_{ik} > 0$ is the k -th edge length. The direction code $b_{ik} \in \{00, 01, 10, 11\}$ is a two-bit binary

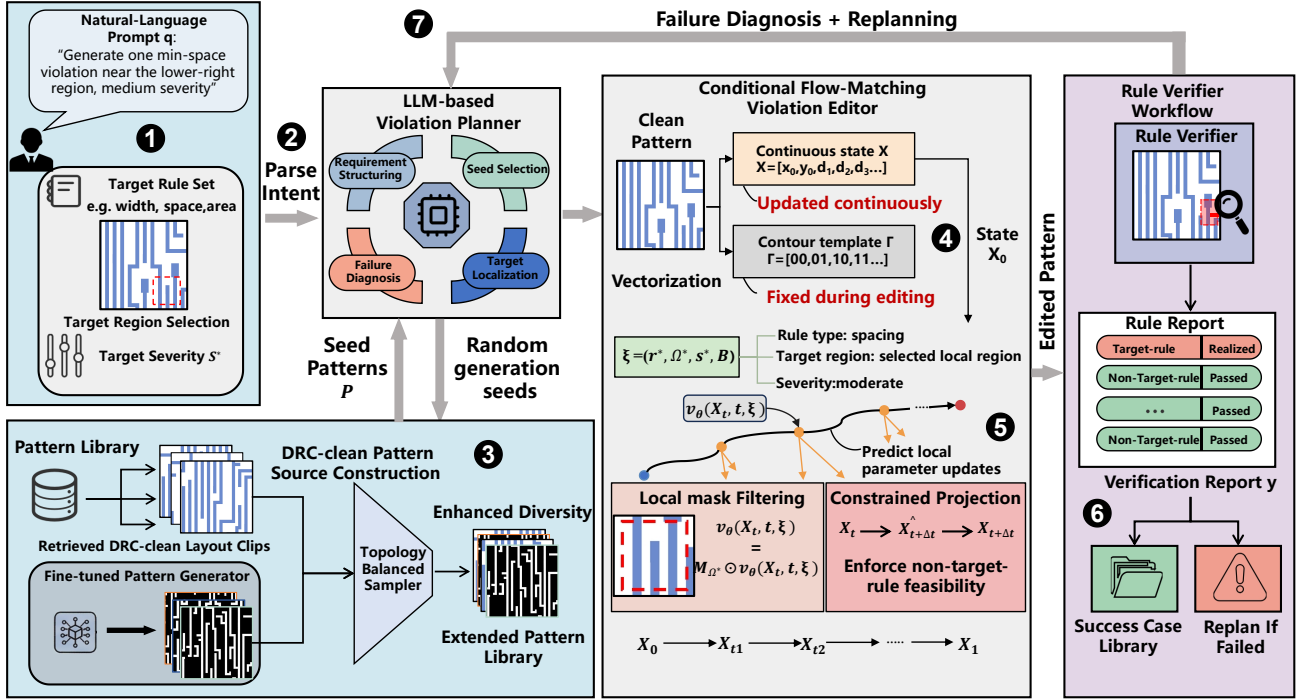


Figure 3: Overview of VioGen. ❶ User specifies target rule, region, and severity via natural language. ❷ LLM-based planner parses intent into structured specification ξ . ❸ Clean seed patterns are retrieved, generated, and topology-balanced. ❹ Pattern is vectorized into continuous state X and contour template Γ . ❺ Flow-matching editor performs masked ODE integration with constraint projection. ❻ Rule verifier checks target activation and non-target compliance. ❼ Failed cases trigger failure diagnosis and replanning through the planner.

encoding of the four Manhattan directions, mapped via $e(b_{ik}) \in \{(\pm 1, 0), (0, \pm 1)\}$.

Compared with the symbolic direction tokens adopted in LayoutTransformer [11], the two-bit binary encoding provides three advantages for downstream editing. First, direction arithmetic reduces to bitwise operations: the opposite direction is obtained by flipping the most significant bit, and perpendicularity is determined via XOR, which simplifies constraint checking during projected sampling. Second, the fixed two-bit width enables uniform tensor packing without variable-length tokenization, thereby reducing padding overhead during batch training. Third, direction codes and edge lengths are explicitly separated in the sequence, allowing the editor to modify continuous geometric parameters while preserving the discrete contour structure. We decompose each pattern as $(X, \Gamma) = \Psi(P)$, where $X \in \mathbb{R}^D$ with $D = \sum_{i=1}^N (2 + K_i)$ contains normalized start-point coordinates and edge lengths, and Γ preserves the direction-code sequence. During editing, only X is updated while Γ remains fixed, ensuring that the Manhattan contour topology is maintained throughout the generation process.

3.3 Agentic Violation Planner

The planner serves as the bridge between high-level user intent and low-level geometric editing. Rather than directly predicting polygon parameters, it interprets natural-language instructions, resolves missing task arguments, and orchestrates the violation generation and verification workflow.

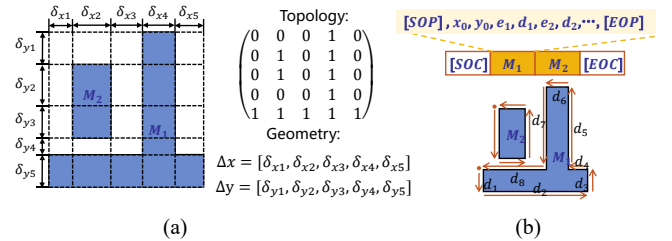


Figure 4: (a) Squish pattern representation based on a binary topology matrix and geometry vectors. (b) Vectorized pattern representation encoding each polygon as a start point and a direction-length sequence with two-bit direction codes.

Requirement Structuring. Given a user prompt q , the planner produces a structured task specification ξ as defined in Eq. (6), thereby reducing the ambiguity of natural-language requests and rendering downstream tool invocation explicit and reproducible. The planner is equipped with a system instruction that specifies the available tools (pattern retrieval, vectorization, editor invocation, and rule verification), the expected output format for ξ , and few-shot examples of requirement-to-specification mappings.

Task Planning and Execution. Upon completion of requirement structuring, the planner decomposes the overall task into a sequence of executable stages: clean pattern selection or generation, vectorization, target-region localization, geometric editing,

and rule verification. A typical generation flow is expressed as $\bar{P}, y = \mathcal{V}(\mathcal{E}(P, \xi), \mathcal{R})$. For context-dependent rules such as EOL spacing, the planner additionally identifies rule-sensitive contour structures within the target region prior to invoking the back-end editor. For instance, rather than treating the target as a generic spatial box, the planner localizes a candidate line-end pair whose relative configuration is most conducive to activating the requested EOL spacing violation at the desired severity.

Failure Diagnosis and Replanning. Targeted violation generation is susceptible to failure, as local edits may induce cascaded violations. To improve robustness, VioGen incorporates a feedback-driven replanning mechanism. Given a failed verification report y , the planner diagnoses whether the failure stems from insufficient target activation, inappropriate localization, or excessive non-target leakage, and revises the specification accordingly via $\xi' = \mathcal{U}(\xi, y)$. The updated specification is then used to initiate another generation attempt. This mechanism reuses verification feedback rather than discarding failed cases, thereby improving sample efficiency.

3.4 Flow-Matching Violation Editor

The violation editor constitutes the core back-end component of VioGen, transforming a clean vectorized pattern into a target-conditioned violation pattern through continuous geometric editing. Building upon the conditional flow-matching formulation introduced in Section 2.2, the editing process is instantiated in the normalized vectorized pattern space defined in Section 3.2, where the clean and target-violation patterns serve as the source and target states, respectively.

For an input clean pattern, the editor operates on the decomposed representation $(X, \Gamma) = \Psi(P)$, modifying only the continuous state X while keeping the contour template Γ fixed. The final violation pattern is reconstructed via $\bar{P} = \Psi^{-1}(X; \Gamma)$. This explicit contour-level access is particularly important for context-dependent rules such as EOL spacing.

Flow-Matching Editing Dynamics. Let $\xi = (r^*, \Omega^*, s^*, \mathcal{B})$ denote the structured target specification produced by the planner. Conditioned on ξ , the editor learns a velocity field $v_\theta(X_t, t, \xi)$ that governs the continuous transformation from the clean state X_0 to the target violation state. Given paired clean and target-violation samples (X_0, X_1^*) , we employ a linear interpolation bridge $X_t = (1 - t)X_0 + tX_1^*$ and train the editor with the conditional flow-matching objective,

$$\mathcal{L}_{\text{edit}} = \mathbb{E}_{(X_0, X_1^*), t \sim \mathcal{U}(0,1)} \|v_\theta(X_t, t, \xi) - (X_1^* - X_0)\|_2^2. \quad (10)$$

At inference time, conditional flow matching alone does not guarantee the absence of non-target-rule failures. To enforce violation isolation, the learned velocity field is augmented with localized masking and constraint projection at each integration step. Let M_{Ω^*} denote a binary mask that restricts edits to polygons within the target region, and let C_{-r^*} denote the feasible set induced by all non-target rules. Algorithm 1 details the complete procedure.

At each integration step, the velocity field is masked to the target region (line 4), an unconstrained Euler update is computed (line 5), and the result is projected onto the non-target feasible set (line 6). For basic rules such as minimum width and spacing, the projection reduces to clamping the corresponding edge length or inter-polygon

Algorithm 1 DRV Case Generation via Projected Flow Matching

Require: Clean state X_0 , contour template Γ , target specification ξ , local mask M_{Ω^*} , non-target set C_{-r^*} , number of steps K

Ensure: Generated violation pattern $\bar{P}$

```

1:  $X \leftarrow X_0, \Delta t \leftarrow 1/K$ 
2: for  $k = 0, \dots, K - 1$  do
3:    $t_k \leftarrow k \cdot \Delta t$ 
4:    $\hat{v} \leftarrow M_{\Omega^*} \odot v_\theta(X, t_k, \xi)$  ▷ Localized velocity
5:    $\hat{X} \leftarrow X + \Delta t \hat{v}$  ▷ Unconstrained Euler step
6:    $X \leftarrow \Pi_{C_{-r^*}}(\hat{X})$  ▷ Non-target projection
7: end for
8:  $\bar{P} \leftarrow \Psi^{-1}(X; \Gamma)$  ▷ Reconstruct pattern
9: return  $\bar{P}$ 

```

distance to the respective rule threshold. For EOL spacing, the projection operates on the joint configuration of a line-end pair: the tentative end-to-end distance is clamped to $\max(\widehat{s}_{\text{eol}}, \tau_{\text{eol}})$, and the resulting correction is distributed proportionally between the two responsible endpoints to maintain contour topology. Upon completion of the final integration step, the violation pattern is reconstructed from the edited state X_K and the preserved contour template Γ (line 8).

3.5 Rule-Ratio Normalization for Transfer

A fundamental difficulty in DRC-related learning is that rule thresholds vary across rule decks. If the editor learns geometric edits in absolute coordinate space, its behavior tends to overfit to the thresholds observed during training. To improve generalization, VioGen introduces a rule-ratio normalization scheme that explicitly decouples geometric editing priors from specific rule values.

For a geometric quantity $z_r(P)$ associated with rule r and threshold τ_r , the ratio-normalized form is defined as

$$\bar{z}_r(P) = \frac{z_r(P)}{\tau_r}. \quad (11)$$

Similarly, for a rule margin $m_r(P) = z_r(P) - \tau_r$, the normalized margin is

$$\bar{m}_r(P) = \frac{m_r(P)}{\tau_r}. \quad (12)$$

This normalization converts absolute geometric values into relative quantities with respect to the legal boundary, enabling the editor to learn proportional contour adjustments rather than memorizing fixed nanometer-scale displacements. For instance, if a minimum-spacing threshold changes from 50nm to 40nm across two rule settings, a 10nm spacing reduction corresponds to $s = 0.20$ in the former but $s = 0.25$ in the latter. By conditioning on the normalized severity rather than absolute displacement, the editor acquires a threshold-invariant editing prior that transfers naturally across settings.

Since both geometry features and severity are normalized by rule thresholds, the editor learns a shared editing prior in ratio space. In the zero-shot setting, adaptation to unseen rule thresholds is achieved by substituting the normalization factors in the input condition. In the few-shot setting, only a small subset of parameters

requires tuning:

$$\theta = \theta_{\text{shared}} \cup \theta_{\text{adapt}}, \quad \frac{|\theta_{\text{adapt}}|}{|\theta|} \ll 1. \quad (13)$$

This design substantially reduces the adaptation cost when the editor encounters unseen rule thresholds.

4 EXPERIMENTAL RESULTS

4.1 Experimental Setup

Datasets. We construct the experimental dataset from two complementary sources. The first source is the ICCAD 2012 contest benchmark [23], from which we extract DRC-clean Manhattan layout clips covering diverse local topologies and geometric complexities. The second source is a set of generated clean patterns produced by a Stable Diffusion 3 [22] model fine-tuned on process-node-specific layout clips. This fine-tuned generator serves as a data augmentation tool: by stochastic sampling with varying style and density conditions, it produces additional DRC-clean clips that complement the contest benchmark in topology coverage and geometric diversity. All generated clips are verified by a rule checker before inclusion.

Each clean pattern from the balanced library is converted into the vectorized representation defined in Section 3.2 and then paired with a corresponding violation counterpart. Violation counterparts are constructed by applying controlled geometric perturbations to each clean pattern using a rule-aware editor with DRC verification, ensuring that each pair contains exactly one target violation at the annotated severity. The combined dataset contains 20,000 clean-violation pairs (12,000 from ICCAD 2012 and 8,000 from the fine-tuned generator), covering five violation types: minimum width, minimum spacing, minimum area, short, and EOL spacing. The training/testing split is 70%/30% with no overlap in seed patterns between the two sets.

Rule Settings. For each violation category, we prepare target specifications with different target regions and multiple normalized severity levels (mild, moderate, severe). To evaluate transferability, we construct two rule settings: in-domain with training-time rule thresholds, and cross-threshold with unseen rule values scaled by factors of $0.8\times$ – $1.5\times$ relative to the training configuration.

Baselines. Since no prior method directly addresses targeted violation case generation, we construct four baselines spanning simple to sophisticated violation injection strategies. **Random Perturbation** applies random coordinate offsets to polygon vertices within the target region without rule-aware guidance. **Heuristic Perturbation** applies rule-specific geometric edits matched to the target rule type (e.g., narrowing a polygon for width violations) but without learning or non-target-rule checking. **Generate-then-Repair** first produces a violation pattern using the same flow-matching architecture as VioGen but without constraint projection, then iteratively applies greedy post-hoc repair to remove non-target failures. **Calibre LSG** is an industrial rule-based pattern construction tool (Siemens EDA) that enumerates DRC-clean geometric configurations; we apply the same heuristic perturbation to its output for violation injection.

Training and Inference Details. The violation editor is trained with the objective in Eq. (10) using Adam [24] ($\text{lr} = 1\times 10^{-5}$, batch

size 64, 100 epochs). Inference uses $K = 50$ Euler steps with constraint projection at each step. For few-shot transfer, 10% of editor parameters are tuned on 1000 target-setting samples. The planner uses Qwen2.5-A17B [25] without fine-tuning. All results are averaged over 5 seeds and 1000 samples per setting. Hardware: 4× Intel Xeon Platinum 8468 + NVIDIA H100 80GB.

4.2 Targeted Violation Case Generation

Basic Rule Categories. VioGen consistently achieves the best balance between target activation and non-target-rule preservation across all four basic rule types. Compared with the strongest baseline (Heuristic Perturbation), VioGen improves average TVSR from 73.9% to 93.0% (+19.1%) while reducing average NTLR from 43.3% to 5.3% (−38.0%). Random Perturbation achieves moderate TVSR but suffers from extremely high NTLR, confirming that rule-unaware editing produces widespread cascaded violations. Generate-then-Repair reduces NTLR to 25.3% relative to heuristic methods but at the cost of lower TVSR, because the iterative repair loop sometimes reverses the intended target violation while correcting non-target failures.

Context-Dependent Rules. The right half of Table 3 reports EOL spacing results evaluated under the moderate severity setting, which is more demanding than the severity-averaged EOL results in Table 2. All baselines degrade substantially on this setting: Random Perturbation drops to 18.2% TVSR with 81.5% NTLR, and even Generate-then-Repair achieves only 43.1% TVSR. This is because EOL violation status depends on the relative contour configuration of line-end structures, which none of the baselines explicitly model. Calibre LSG achieves the lowest TVSR (12.4%) on EOL, as its rule-based enumeration engine has no mechanism for deliberately constructing context-dependent violations. VioGen achieves 82.3% TVSR and 7.6% NTLR on EOL, demonstrating that contour-aware editing in vectorized space enables direct manipulation of the responsible line-end structure. Notably, VioGen’s performance gap between basic rules and EOL is only 10.7%, substantially smaller than the gaps observed for Heuristic Perturbation (−38.3%) and Generate-then-Repair (−26.3%).

Severity Controllability. The left half of Table 3 reports SCE across three severity levels. VioGen maintains consistently low SCE, while Random Perturbation shows the highest error because it cannot calibrate the magnitude of geometric edits. Calibre LSG, despite being an industrial tool, shows high SCE because it is designed for clean pattern construction rather than controlled violation injection. Heuristic Perturbation improves by matching edit type to the target rule, but still lacks precise severity control. Generate-then-Repair achieves moderate SCE, though the repair loop may over-correct the violation and shift the realized severity away from the requested level. All methods show lower SCE at the severe level than at mild, because severe violations allow larger geometric margins and thus are more tolerant to imprecise edits. Fig. 5(a)–(e) further illustrate that VioGen produces controlled severity variation across rule types without excessive disturbance to surrounding contour structures.

4.3 Diversity and Efficiency Analysis

Diversity of Generated Violation Library. Table 4 reports H_1 , H_2 , and per-pattern generation time for VioGen and the baselines. VioGen achieves $H_1 = 8.76$ and $H_2 = 12.32$, the highest entropy

Table 2: Main results and ablation study on targeted single-violation case generation. Higher TVSR, lower NTLR, and lower SCE indicate better performance.

Method	Min-width		Min-space		Min-area		Short		EOL		Average		
	TVSR $\uparrow$	NTLR $\downarrow$	TVSR $\uparrow$	NTLR $\downarrow$	TVSR $\uparrow$	NTLR $\downarrow$	TVSR $\uparrow$	NTLR $\downarrow$	TVSR $\uparrow$	NTLR $\downarrow$	TVSR $\uparrow$	NTLR $\downarrow$	SCE $\downarrow$
Random Perturbation	72.3	61.5	68.7	65.2	74.1	58.3	63.8	68.7	41.2	72.4	64.0	65.2	0.34
Heuristic Perturbation	81.6	38.4	78.2	42.1	82.5	35.7	74.3	46.8	52.7	53.6	73.9	43.3	0.24
Generate-then-Repair	76.4	22.1	73.8	25.6	78.2	19.8	70.5	27.3	48.3	31.5	69.4	25.3	0.17
Calibre LSG	65.3	52.4	61.8	56.1	67.5	49.3	58.2	59.6	12.4	72.8	53.0	58.0	0.31
VioGen w/o Planner	89.3	6.8	87.5	7.4	90.1	5.9	85.6	8.3	76.2	11.5	85.7	8.0	0.08
VioGen w/o Projection	93.1	24.7	91.8	27.3	94.2	22.1	90.5	29.6	85.4	33.8	91.0	27.5	0.09
VioGen w/o RatioNorm	92.5	7.2	90.6	8.1	93.4	6.5	89.1	9.4	83.7	12.3	89.9	8.7	0.10
VioGen w/o LocalMask	90.8	14.3	88.4	16.7	91.6	12.8	87.2	18.5	80.1	21.4	87.6	16.7	0.11
VioGen	95.2	4.1	93.6	4.8	96.1	3.5	92.3	5.7	87.8	8.6	93.0	5.3	0.05

Table 3: Severity controllability and EOL spacing results. Lower SCE indicates better severity control.

Method	SCE $\downarrow$			EOL Results		
	Mild	Mod.	Severe	TVSR $\uparrow$	NTLR $\downarrow$	SCE $\downarrow$
Random Perturb.	0.43	0.38	0.34	18.2	81.5	0.46
Heuristic Perturb.	0.27	0.21	0.18	35.6	64.3	0.30
Gen.-then-Repair	0.22	0.17	0.15	43.1	49.7	0.24
Calibre LSG	0.35	0.30	0.26	12.4	72.8	0.41
VioGen w/o Proj.	0.13	0.10	0.08	70.5	36.2	0.13
VioGen	0.07	0.05	0.04	82.3	7.6	0.06

Note. EOL results are evaluated under the moderate severity setting. Calibre LSG is included as a reference industrial tool not designed for targeted violation generation.

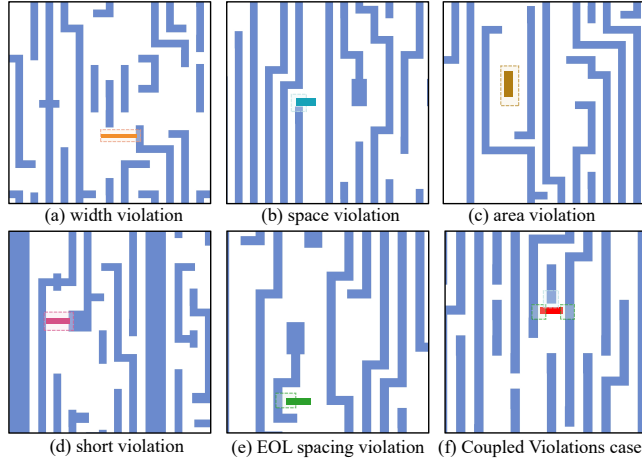


Figure 5: Qualitative results of VioGen. (a)–(e) show isolated single-type violations for width, spacing, area, short, and EOL spacing rules. (f) shows a coupled-violation case from a baseline where one intended edit triggers multiple unintended failures.

on both diversity metrics among all evaluated methods. Heuristic Perturbation produces the lowest diversity ($H_1 = 2.53$, $H_2 = 1.94$) because each rule type employs a single fixed editing template, leading to repeated geometric patterns. Generate-then-Repair exhibits

Table 4: Diversity and runtime comparison.

Method	$H_1\uparrow$	$H_2\uparrow$	Time (s) $\downarrow$
PatternPaint [13] [†]	4.11	12.01	1.02
Calibre LSG	2.85	3.29	0.37
Random Perturbation	3.12	2.61	0.01
Heuristic Perturbation	2.53	1.94	0.05
Generate-then-Repair	3.35	2.78	8.3
VioGen	8.76	12.32	2.1

[†] Cited from published results.

moderate diversity but lower $H_2 = 2.78$, as the repair loop tends to converge to similar correction patterns. Calibre LSG achieves $H_1 = 2.85$ and $H_2 = 3.29$, reflecting the limited topological variety of its rule-based enumeration engine.

We note that the SOTA PatternPaint [13] reports $H_1 = 4.11$ and $H_2 = 12.01$ on a different experimental setting (Intel 18A PDK with pixel-level squish representation), making direct numerical comparison non-trivial due to differences in dataset, pattern representation, and library scale. Nevertheless, VioGen achieves substantially higher H_1 and comparable H_2 relative to PatternPaint, indicating that violation-aware editing in vectorized space does not sacrifice pattern diversity, and that the topology-balanced sampling step contributes to broad topological coverage.

Effect of ODE Integration Steps. Fig. 6 shows how the number of integration steps K affects TVSR, NTLR, and per-pattern generation time. With too few steps ($K \leq 10$), the trajectory approximation is inaccurate and TVSR drops below 80% while NTLR exceeds 18%. Beyond $K = 50$, performance saturates while runtime grows linearly from 2.1s to 7.8s. We use $K = 50$ as the default setting, which achieves near-optimal generation quality at a per-pattern cost of 2.1 seconds.

Runtime. As shown in the last column of Table 4, VioGen generates a violation case in 2.1 seconds on average, including vectorization, K -step ODE integration, and constraint projection. Random and Heuristic Perturbation are the fastest but at the cost of poor violation quality (Table 2). Generate-then-Repair is the slowest at 8.3

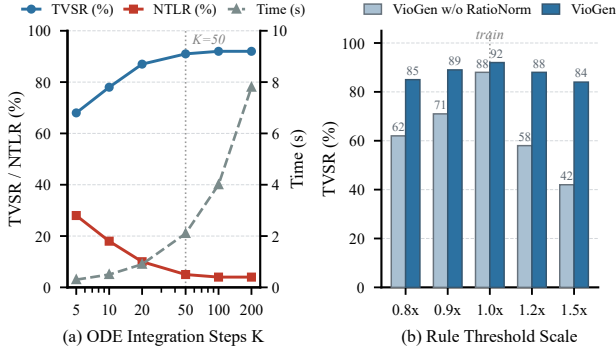


Figure 6: (a) Effect of ODE integration steps K on TVSR, NTLR, and per-pattern generation time. Performance saturates at $K = 50$ while runtime grows linearly. (b) Zero-shot TVSR under varying rule threshold scales. Rule-ratio normalization maintains stable performance across unseen thresholds.

seconds due to its iterative repair loop, which requires multiple DRC checks per pattern. VioGen is 4× faster than Generate-then-Repair while achieving substantially better TVSR and NTLR, demonstrating a practical balance between generation quality and efficiency.

4.4 Transferability

Zero-Shot Transfer. We evaluate the editor on rule thresholds not seen during training without any parameter update. VioGen achieves a zero-shot TE of 86.2%, while VioGen w/o RatioNorm drops to 57.8%, confirming that rule-ratio normalization substantially improves performance retention under unseen thresholds. Fig. 6 further illustrates this effect: as the threshold scale deviates from the training setting (1.0×), VioGen w/o RatioNorm degrades sharply while VioGen remains stable.

Few-Shot Adaptation. Under a limited adaptation budget (10% parameters, 1000 samples), VioGen recovers to 90.5% TE compared with 71.3% for VioGen w/o RatioNorm. This confirms that the ratio-normalized editing prior transfers effectively across rule settings with minimal adaptation cost.

4.5 Ablation Study

We additionally compare with four ablated variants of VioGen: **w/o Planner**, **w/o Projection**, **w/o RatioNorm**, and **w/o Local-Mask**. The ablation results are reported in the lower half of Table 2 (Average columns).

Constraint-Projected Sampling is the most critical component for violation isolation: removing it causes the largest NTLR increase of 22.2% absolute, confirming that stepwise feasibility enforcement during ODE integration is essential for suppressing cascaded non-target violations.

Local Mask removal increases NTLR by 11.4% absolute and SCE by 0.06, as global editing introduces unnecessary disturbance to contour regions far from the target, leading to both collateral rule failures and degraded severity precision.

Rule-Ratio Normalization removal has moderate impact on in-domain performance (SCE increases by 0.05) but mainly degrades

transfer performance as discussed in Section 4.4, where zero-shot TE drops by 28.4%.

Planner removal hurts cases requiring multiple attempts: TVSR drops by 7.3%, while NTLR is less affected by 2.7%, confirming that the planner primarily improves target activation through failure-aware replanning rather than directly affecting violation isolation.

4.6 Evaluation of LLM-based Planner

Requirement Structuring. Given a natural-language request, the planner outputs a structured specification:

```
# Target Specification
## Target rule: EOL spacing
## Target region: upper-right local contour
## Target severity: moderate
## Other rules: preserve non-target-rule compliance
## Sample count: n
```

Failure-Aware Replanning. When verification fails, the planner revises the target specification based on feedback and re-launches editing. Fig. 5(f) illustrates a typical failure mode that triggers replanning: a baseline edit intended to activate a single target rule instead produces coupled violations across multiple rules, which VioGen’s planner detects and corrects through specification revision.

Quantitative Analysis. Among all generated cases, 76.4% succeed on the first editing attempt without replanning. For the remaining 23.6% failed cases, the planner recovers 81.2% through feedback-driven replanning within an average of 1.7 additional iterations. The most common failure mode triggering replanning is non-target leakage (52.3% of all failures), followed by insufficient target activation (31.5%) and incorrect target localization (16.2%). These statistics confirm that the planner substantially improves overall generation success from 76.4% to 95.6%. Note that the 76.4% first-attempt rate reflects single-pass editing success before any replanning, whereas the 93.0% TVSR reported in Table 2 is the final rate measured over the complete pipeline including cases where replanning still fails, and that non-target leakage is the dominant bottleneck in challenging cases.

Discussion. The planner serves as a task coordinator rather than merely a natural-language interface, improving robustness through feedback-driven replanning in challenging cases where single-pass editing is insufficient.

5 CONCLUSION

We have presented VioGen, a framework for targeted single-type DRC violation case generation from natural-language instructions. VioGen integrates an LLM-based violation planner with a conditional flow-matching editor that performs contour-aware geometric editing in normalized vectorized pattern space. The constraint-projected sampling scheme suppresses non-target-rule violations at each ODE integration step, achieving 93.0% average TVSR with only 5.3% non-target leakage across five rule categories. The rule-ratio normalization mechanism enables 86.2% zero-shot transfer efficiency under unseen rule thresholds. By extending layout pattern generation from DRC-clean output to controllable violation-aware case construction, VioGen provides a practical tool for DRC tool validation and hotspot benchmarking.

REFERENCES

- [1] Yen-Ting Yu, Ya-Chung Chan, Saurabh Sinha, Iris Hui-Ru Jiang, and Charles Chiang. DRC-based hotspot detection considering edge tolerance and incomplete specification. In *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 67–74, 2014.
- [2] Gaurav R. Reddy, Constantinos Xanthopoulos, and Yiorgos Makris. Enhanced hotspot detection through synthetic pattern generation and design of experiments. In *Proc. IEEE VLSI Test Symposium (VTS)*, pages 1–6, 2018.
- [3] Ran Chen, Wei Zhong, Haoyu Yang, Hao Geng, Xuan Zeng, and Bei Yu. Faster region-based hotspot detection. In *Proc. ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2019.
- [4] Haoyu Yang, Piyush Pathak, Frank Gennari, Ya-Chieh Lai, and Bei Yu. Automatic layout generation with applications in machine learning engine evaluation. In *Proc. SPIE Design-Process-Technology Co-optimization for Manufacturability*, volume 10962, 2019.
- [5] Rasit O. Topaloglu. ICCAD-2014 CAD contest in design for manufacturability flow for advanced semiconductor nodes and benchmark suite. In *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 602–606, 2014.
- [6] Andrew B. Kahng, Chul-Hong Park, and Xu Xu. Fast dual graph based hotspot detection. In *Proc. SPIE Photomask Technology*, volume 6349, pages 628–635, 2006.
- [7] Xiang Fang, Shou-Yuan Ma, Chien-Nan Lin, Hsu-Tang Liu, Chuan-Chun Lee, Chuen-Huei Yang, I-Y. Chang, Erwin Deng, Ming-Feng Shen, Min-Ying Lu, et al. Achieve high hotspot detection accuracy by pattern scoring. In *Proc. SPIE Photomask Japan*, volume 10807, 2018.
- [8] Haoyu Yang, Piyush Pathak, Frank Gennari, Ya-Chieh Lai, and Bei Yu. DeePattern: Layout pattern generation with transforming convolutional auto-encoder. In *Proc. ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2019.
- [9] Zixiao Wang, Yunheng Shen, Wenqian Zhao, Yang Bai, Guojin Chen, Farzan Farnia, and Bei Yu. DiffPattern: Layout pattern generation via discrete diffusion. In *Proc. ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2023.
- [10] Xiaopeng Zhang, James Shiely, and Evangeline F.Y. Young. Layout pattern generation and legalization with generative learning models. In *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–9, 2020.
- [11] Li Wen, Yibo Zhu, Lei Ye, Guojin Chen, Bei Yu, Jianguo Liu, and Chuan Xu. LayoutTransformer: Generating layout patterns with transformer via sequential pattern modeling. In *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–9, 2022.
- [12] Wenqian Zhao, Zixiao Wang, Yunheng Shen, Yang Bai, Guojin Chen, and Bei Yu. ControlLayout: Controllable layout pattern generation. In *Proc. ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2024.
- [13] Guanglei Zhou, Chandra Korrapati, Gangadhara Reddy, Chih-Pin Chang, Jianli Pan, Jiang Hu, Yiran Chen, and Dipto G. Thakurta. PatternPaint: Practical layout pattern generation using diffusion-based inpainting. In *Proc. ACM/IEEE Design Automation Conference (DAC)*, pages 1–7, 2025.
- [14] Guanglei Zhou, Dipto G. Thakurta, Jiang Hu, and Yiran Chen. Vario: Generating diverse layout variations with variational diffusion. In *Proc. SPIE Advanced Lithography + Patterning*, volume 12955, 2024.
- [15] Zhuolun Wu, Shuaiwen Leon Liang, et al. ChatEDA: A large language model powered autonomous agent for EDA. In *Proc. Machine Learning for CAD (MLCAD)*, pages 1–6, 2024.
- [16] Ruizhe Zhong et al. Llm4eda: Emerging progress in large language models for electronic design automation. *arXiv preprint arXiv:2401.12224*, 2024.
- [17] Zixiao Wang, Yunheng Shen, Wenqian Zhao, Yang Bai, Guojin Chen, Farzan Farnia, and Bei Yu. ChatPattern: Layout pattern customization via natural language. In *Proc. ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2024.
- [18] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *Proc. International Conference on Learning Representations (ICLR)*, 2023.
- [19] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *Proc. International Conference on Learning Representations (ICLR)*, 2023.
- [20] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pages 6840–6851, 2020.
- [21] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *Proc. International Conference on Learning Representations (ICLR)*, 2021.
- [22] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorber, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. *arXiv preprint arXiv:2403.03206*, 2024.
- [23] J. Andres Torres and ICCAD Contest Committee. ICCAD 2012 CAD contest: Fuzzy pattern matching for physical verification. In *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2012.
- [24] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proc. International Conference on Learning Representations (ICLR)*, 2015.
- [25] An Yang, Baosong Yang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.